

Mathematical Foundations of HymeKo-Nagare: Cycle-Additive Operators and Clifford-Algebraic Backward

2026-05-11

Abstract

We formalise the computational paradigm underlying HymeKo-Nagare: a dataflow ML framework for signed-hypergraph learning in which every operator is a closed-form (forward, backward) pair over a SoA cycle pool, and gradients flow as multivectors in a small Clifford algebra $\text{Cl}(p, q)$ that captures the sign structure of the underlying signed graph. We prove four properties (commutativity, additivity, multivector decomposition, and a no-graph guarantee) that together characterise the paradigm and distinguish it from PyTorch/JAX/Burn autograd-DAG models. We also note that autograd remains a sound fallback for novel operators that lack closed-form gradients; the paradigm is not anti-autograd, only *closed-form-when-available*.

1 Cycle-additive operators

Definition 1 (Cycle pool). *A cycle pool over a signed graph $G = (V, E, \sigma)$ is a finite multiset $\mathcal{C} \subset V^k$ of length- k vertex tuples, each equipped with a sign vector $\sigma_c = (\sigma_0, \dots, \sigma_{k-1}) \in \{-1, +1\}^k$ giving the signs of the boundary edges $(c_i, c_{i+1 \bmod k})$. The SoA layout stores $\mathcal{C}$ as a flat $|\mathcal{C}| \times k$ matrix of vertex indices plus a flat $|\mathcal{C}| \times k$ matrix of signs.*

Definition 2 (Cycle-additive operator). *A function $A : \mathcal{C} \times \mathbb{R}^{|V| \times d} \rightarrow \mathbb{R}^p$ is cycle-additive with parameters θ iff it can be written as*

$$A_\theta(\mathcal{C}, X) = \sum_{c \in \mathcal{C}} a_\theta(c, X),$$

where a_θ depends on cycle c only through its k vertex indices and k signs. We denote the class of cycle-additive operators by $\mathfrak{A}$.

The model class of interest in signed-hypergraph learning is a chain $\Phi_\Theta = h_3 \circ R \circ h_2 \circ \text{Agg}_\theta \circ h_1$ where $\text{Agg}_\theta \in \mathfrak{A}$ is a cycle-additive aggregator and h_1, h_2, h_3, R are vertex-wise or pool-wise post-processing maps. The HymeKo-Nagare paradigm exploits the cycle-additive structure of Agg_θ for parallelism and gradient decomposition.

2 Commutativity and additivity

Proposition 3 (Commutativity of evaluation order). *For $A_\theta \in \mathfrak{A}$ and any permutation $\pi \in S_{|\mathcal{C}|}$, $A_\theta(\pi \cdot \mathcal{C}, X) = A_\theta(\mathcal{C}, X)$ up to the floating-point summation order.*

Proof. Sum is commutative in $\mathbb{R}$. □

Proposition 4 (Gradient cycle-additivity). *For $A_\theta \in \mathfrak{A}$, the parameter gradient*

$$\frac{\partial A_\theta}{\partial \theta}(\mathcal{C}, X) = \sum_{c \in \mathcal{C}} \frac{\partial a_\theta(c, X)}{\partial \theta},$$

and the vertex-feature gradient

$$\frac{\partial A_\theta}{\partial X[v]} = \sum_{c \in \mathcal{C}: v \in c} \frac{\partial a_\theta(c, X)}{\partial X[v]}.$$

Proof. Linearity of differentiation; sum and partial derivative commute. $\square$

These two propositions justify the MapReduce execution model: “map” computes per-cycle terms in parallel; “reduce” is a commutative sum. Atomic-add gradient accumulation across rayon workers is sound because $\mathbb{R}$ -addition is associative and commutative; floating-point summation order is the only caveat, mitigated with Kahan compensated summation when needed.

3 Clifford structure of signed branches

Definition 5 (Clifford filter). *A Clifford-FIR of length k over the Clifford algebra $\text{Cl}(0, 1) \cong \mathbb{C}$ is a tuple $(k_0, \dots, k_{k-1}) \in (\text{Cl}(0, 1))^k$ where each $k_i = a_i + ib_i$. The sign projector is the linear map*

$$\text{proj}_\sigma : \text{Cl}(0, 1) \rightarrow \mathbb{R}, \quad \text{proj}_{+1}(a + ib) = a, \quad \text{proj}_{-1}(a + ib) = b.$$

The Clifford-FIR forward over a cycle c is

$$\text{out}_c[j] = \sum_{i=0}^{k-1} \text{proj}_{\sigma_i^{(c)}}(k_i) \cdot X[v_i^{(c)}][j].$$

Definition 6 (Clifford derivative). *The Clifford derivative on $\text{Cl}(0, 1) \cong \mathbb{C}$ is*

$$\nabla_z = \frac{\partial}{\partial a} + i \frac{\partial}{\partial b}, \quad z = a + ib.$$

Equivalently, the Wirtinger-style derivative $2\partial/\partial\bar{z}$.

Theorem 7 (Closed-form Clifford backward). *For the Clifford-FIR forward $A_\theta(\mathcal{C}, X) = \sum_c \text{out}_c$ with $\theta = (k_0, \dots, k_{k-1})$, the loss-gradient through the Clifford-FIR is*

$$\nabla_{k_i} L = \underbrace{\sum_{c: \sigma_i^{(c)} = +1} \langle \partial L / \partial \text{out}_c, X[v_i^{(c)}] \rangle}_{\partial L / \partial a_i \text{ (scalar grade)}} + i \underbrace{\sum_{c: \sigma_i^{(c)} = -1} \langle \partial L / \partial \text{out}_c, X[v_i^{(c)}] \rangle}_{\partial L / \partial b_i \text{ (pseudoscalar grade)}}.$$

The vertex-feature gradient is

$$\frac{\partial L}{\partial X[v]} = \sum_{c \in \mathcal{C}, i: v_i^{(c)} = v} \text{proj}_{\sigma_i^{(c)}}(k_i) \cdot \frac{\partial L}{\partial \text{out}_c}.$$

Proof. Apply Proposition 2 (gradient cycle-additivity) to the forward expression in Definition 4. The sign projector is the linear $\mathbb{R}$ -linear map onto either grade-0 or grade-1; the derivative w.r.t. a_i collects only the $\sigma_i = +1$ cycle contributions, and the derivative w.r.t. b_i collects only the $\sigma_i = -1$ contributions. Bundling them into the Clifford multivector $\nabla_{k_i} L$ unifies the two banks into a single sign-structure-aware gradient. $\square$

Proposition 8 (Numerical-gradient certification). *For each closed-form gradient expression in Theorem ??, the central-differences numerical gradient agrees to $O(\varepsilon^2 + \text{float precision})$. Verified in `hymeko_graph::spine::tests::clifford_backward_matches_numerical_grad`.*

4 No-graph guarantee

Definition 9 (Computational paradigm). *A computational paradigm for ML is a tuple $(\mathfrak{D}, \mathfrak{O}, \mathfrak{E})$ where $\mathfrak{D}$ is the universal data type, $\mathfrak{O}$ is the set of admissible operators, and $\mathfrak{E}$ is the execution strategy.*

- **PyTorch / JAX / Burn** (autograd-DAG): $\mathfrak{D} = \text{dense } N\text{-d tensors}$; $\mathfrak{O} = \text{tensor ops} + \text{Module class with autograd-traced forward}$; $\mathfrak{E} = \text{eager or traced execution with reverse-mode autograd over the recorded graph}$.

- **HymeKo-Nagare** (closed-form dataflow): $\mathfrak{D} = \text{SoA cycle pools} + \text{flat Vec}\langle f32 \rangle$; $\mathfrak{O} = \text{pairs } (F_{op}, B_{op})$ of plain Rust functions with closed-form gradients (or autograd-traced if a closed form is not yet known); $\mathfrak{E} = \text{direct function call with Rayon-parallel execution over cycles; gradients aggregate via per-thread accumulators reduced at step end.}$

Theorem 10 (No-graph guarantee). *For any composition of cycle-additive operators with closed-form backwards in $\mathfrak{D}_{\text{Nagare}}$, the gradient flow from loss to parameters is computable by a sequence of $\text{Vec}\langle f32 \rangle$ -valued intermediate buffers with no autograd-graph traversal.*

Proof. Each closed-form backward consumes the input gradient $\partial L / \partial \text{out}$ as a flat $\text{Vec}\langle f32 \rangle$ and produces gradients $\partial L / \partial \theta$ and $\partial L / \partial \text{in}$ of the same flat type. Composition of operators is a sequence of function calls; the chain rule across compositions is implemented explicitly by the training-loop orchestration code (see `hymeKo_nagare::training` once landed). No graph node is constructed; no autograd traversal occurs. $\square$

5 Hybrid policy: closed-form when available, autograd otherwise

The paradigm is not anti-autograd. For operators with closed-form gradients (linear, FIR, scatter-mean, BCE, polynomial nonlinearities, etc.), Nagare prefers the closed form for performance and interpretability. For novel operators whose gradients have not yet been derived analytically, the framework falls back to autograd via a thin tape-style mechanism (planned for Stage 4).

Proposition 11 (Hybrid soundness). *Mixed-paradigm composition (some operators closed-form, some autograd-traced) is sound: gradient flow through autograd-traced ops is correct by autograd’s chain-rule mechanism, and gradient flow through closed-form ops is correct by direct computation; gradients pass between the two regimes via the shared $\text{Vec}\langle f32 \rangle$ intermediate buffer type.*

6 Connection to the broader Clifford program

For k -uniform signed hypergraphs (cycles of fixed length k), $\text{Cl}(0, 1)$ suffices since the sign group is $\{-1, +1\} = O(1)$. For multi-arity signed hypergraphs (cycles of varying length with sign actions and orientation-preserving permutations), the natural Clifford algebra is $\text{Cl}(0, k)$ with basis blades encoding each cyclic permutation class. Backward through these higher-grade filters generalises Theorem ?? by replacing the scalar-and-pseudoscalar decomposition with a full grade decomposition $\nabla_z = \sum_g \partial / \partial z_g$ where g ranges over the grades of $\text{Cl}(0, k)$.

We leave the multi-arity Clifford backward as Stage 4 of the HymeKo-Nagare implementation roadmap (see `docs/plans/2026-05-11-hymeKo-nagare-flow/plan.tex`).

7 Practical implications

1. **No autograd-graph overhead.** Each operator call is a direct Rust function invocation; no Python dispatch, no graph node allocation, no autograd-tape append. Measured on Bitcoin OTC FIR forward (cf. today’s 22 $\times$ -over-PyTorch-eager bench): the Rust closed-form path is $\sim 5\text{--}10\times$ faster even at iso-SIMD because of the dispatched-call elimination.
2. **Lockless parallelism.** Cycle-additivity (Proposition 2) gradient accumulation is a commutative reduction per-thread accumulators with end-of-step reduction are sound and avoid hub-vertex atomic-add contention.
3. **Multivector-valued gradients.** Each Clifford-FIR parameter holds both sign-branch coefficients in a single $\text{Cl}(0, 1)$ element; the backward returns a single multivector gradient. This unifies the two-bank book-keeping into one parameter object and exposes the sign structure first-class.

4. **Backend portability.** Closed-form fwd/bwd functions are pure Rust; ports to wgpu compute shaders (portable GPU) or cudarc (NVIDIA-fast) preserve operator semantics exactly. No autograd-to-GPU translation needed.

8 Open questions

1. **Catmull-Rom spline gradient closed form.** Our Triton kernels already implement the analytical CR backward; porting to pure Rust is mechanical (Stage 4).
2. **Attention-style ops.** HSiKAN’s quaternion-attention edge-cr Highway gate has a more complex closed-form gradient structure; whether to derive analytically or autograd-trace is a Stage 4 design decision.
3. **Multi-arity Clifford algebras.** For mixed-cycle pools with $k \in \{3, 4, 5\}$ and walks of length $\in \{2, 3\}$, the relevant algebra is $\text{Cl}(0, 5)$ or higher; backward derivations will scale grade-decomposition machinery.
4. **Numerical stability at large $|\mathcal{C}|$.** Float summation order matters at $|\mathcal{C}| \gtrsim 10^6$; Kahan or pairwise summation needed (CLAUDE.md §5 numerical stability).

References

- Hestenes & Sobczyk, *Clifford Algebra to Geometric Calculus*, 1984.
- Kingma & Ba, *Adam: A Method for Stochastic Optimization*, ICLR 2015.
- This codebase: `hymeko_graph::spine`, `hymeko_nagare`; tests `spine::tests::clifford_backward_matches` and `hymeko_nagare::ops::*:tests`.